SIMATS ENGINEERING ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics

COURSE CODE: CSA1514

COURSE OUTCOME COVERED

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Dave's Taxonomy: Manipulation (Level 2)

Total Marks: 25

Question Count: 5

Code of Conduct

I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **RASHMITHA .S**

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

Application Based Questions Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

QUESTION 1 – E-COMMERCE AUTO-SCALING AND LOAD BALANCING

Scenario and problem interpretation:

A retail company receives unpredictable surges in customer visits during seasonal promotions. The application must preserve product browsing, shopping carts, checkout and payment confirmation even when demand grows abruptly. An architecture sized permanently for peak traffic would be expensive, while a fixed small deployment would overload during flash sales. The appropriate response is an elastic cloud application tier supported by load balancing, monitoring, caching and resilient data services. The business objective is not merely to keep servers running, but to maintain successful customer transactions with acceptable response times.

Proposed cloud architecture:

The recommended design places a content delivery network and web application firewall in front of a regional application load balancer. Static product images and style assets are served from the CDN, reducing origin traffic. The load balancer distributes dynamic API requests among stateless application instances running across at least two availability zones. Auto-scaling increases or decreases application replicas according to measured demand. A managed database stores orders and inventory, while an in-memory cache accelerates frequently requested catalog information. A durable message queue decouples checkout requests from noncritical email, analytics and notification jobs.

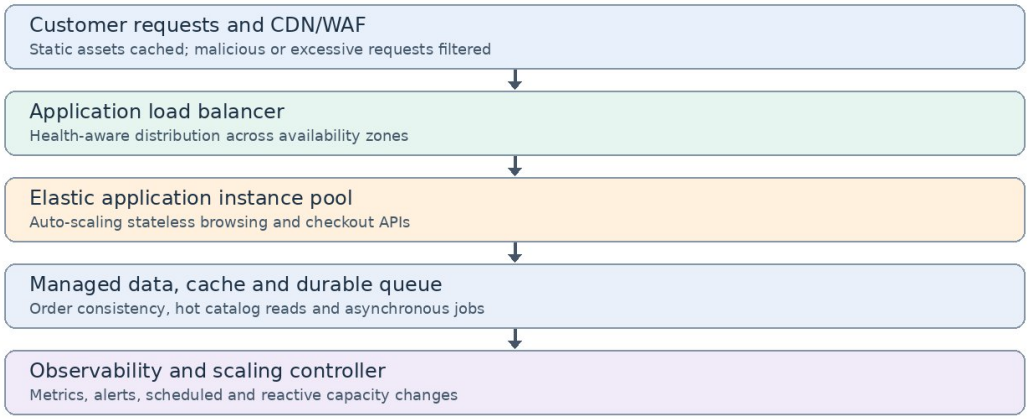


Figure 1. Elastic e-commerce architecture with resilient load balancing and demand-driven scaling.

How load balancing reduces downtime:

A load balancer continuously checks the health of application instances and routes new requests only to healthy targets. If one server fails, traffic can be redirected to remaining healthy instances, provided that sufficient capacity exists. Deploying instances across independent availability zones reduces dependence on one machine or one zone. Connection draining during deployment permits in-flight requests to complete before an instance is removed. Session data should be stored in a shared session store or encoded securely in suitable tokens rather than relying on local server memory. Health checks must distinguish a genuinely healthy application from a process that is running but unable to reach essential dependencies.

How auto-scaling improves performance and customer experience:

Reactive scaling can add instances when CPU, request count per target, queue backlog or measured latency crosses a sustained threshold. Predictive or scheduled scaling should pre-warm capacity before known events such as a festival sale, because new instances take time to initialize. Minimum and maximum replica counts, cooldown periods and scaling hysteresis reduce oscillation. During a sale, additional instances reduce request queuing, help maintain fast product searches and lower the risk of checkout timeouts. When traffic subsides, the application pool can shrink to control cost. Database connection pooling, rate limits and queue backpressure are essential to prevent a scaled-out application tier from overwhelming downstream services.

Monitoring tools and proactive operations:

Cloud monitoring services collect request rate, p95 and p99 latency, HTTP error percentage, load-balancer healthy-target count, instance CPU and memory, database connections, cache hit ratio, payment failures and queue age. Distributed tracing identifies slow dependencies, while structured logs support investigation of failed checkouts. Alerts should reflect customer impact rather than every small metric fluctuation. Synthetic checkout tests can detect broken user journeys before customers report them. Operations teams should run load tests and controlled zone-failure drills to verify that scaling and traffic distribution meet service objectives.

Real-world e-commerce application example:

Consider a hypothetical online retailer whose ordinary traffic is 2,000 requests per second and whose flash-sale demand reaches 12,000 requests per second. These are illustrative design

assumptions, not observed company data. The retailer pre-scales its API pool before the sale, uses a CDN for product media and allows auto-scaling to add capacity as measured request demand rises. Checkout events enter a durable queue when downstream services become slow, while inventory reservation and payment status remain strongly controlled to avoid overselling or duplicate charges. The result sought is sustained transaction success with low latency, rather than simply a high count of running servers.

Table 1. E-commerce scaling controls and acceptance metrics

Control	Purpose	Validation evidence
Load balancer health checks	Exclude failed targets	Healthy target count and error-rate trend
Horizontal auto-scaling	Absorb demand spikes	Replica count and p95 response time
CDN and application cache	Reduce origin/database load	Cache hit ratio and origin traffic
Queue and backpressure	Protect downstream processing	Queue age and completed order count
Synthetic checkout monitoring	Detect user-impacting failures	Successful end-to-end transaction tests

Conclusion:

An elastic, multi-zone architecture combines load balancing, auto-scaling, caching and monitoring to reduce downtime and protect the shopping experience during seasonal sales. The solution succeeds only when capacity planning includes databases, payment gateways and operational failover, not just web servers.

QUESTION 2 – SECURE CLOUD BACKUP STORAGE AND DISASTER RECOVERY

Scenario and protection objectives:

The organization must retain large backup volumes securely while remaining able to recover data after accidental deletion, ransomware, equipment failure or a site outage. Backup storage is different from a live replica: a replica may reproduce corrupted or deleted records immediately, whereas a backup preserves recoverable historical states. The design must balance durability, recoverability, confidentiality, regulatory retention and cost. The required recovery time objective (RTO) and recovery point objective (RPO) should be agreed with business owners before selecting storage and backup schedules.

Cloud storage durability and redundancy architecture:

A suitable architecture uses a managed object storage service for encrypted backup objects, together with scheduled database snapshots and point-in-time recovery where supported. Cloud object storage typically uses erasure coding or replication across failure domains to tolerate underlying device failures; the exact durability commitment depends on the selected service and tier. Versioning protects previous object generations, while retention locks or immutable backup vaults reduce the risk of deliberate or accidental deletion. Cross-account backup separation reduces the impact of a compromised production administrator. A second region may be justified for disaster recovery, but cross-region copies add cost and must meet data-residency requirements.

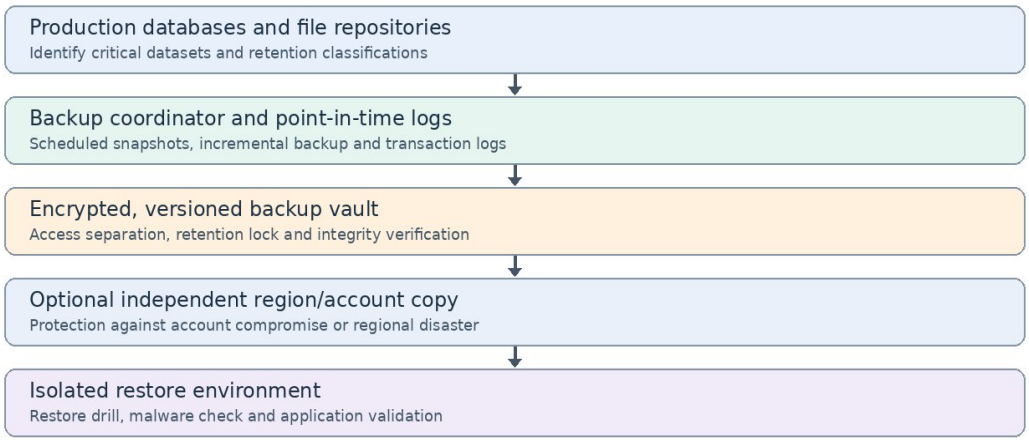


Figure 2. Layered cloud backup architecture separating recoverable copies from production systems.

Encryption and access control:

Backups should be encrypted in transit using modern TLS and at rest using provider-managed or customer-managed keys according to the threat model. Customer-managed key policies should restrict decryption and key administration to approved roles. Key rotation, recovery procedures and deletion protection must be documented, because losing a key can make otherwise intact backups unrecoverable.

Identity and access management should apply least privilege, separate backup operators from production administrators and require multifactor authentication for privileged actions. Backup restore permissions should be especially restricted, since a restore can expose confidential historical records. Central audit logs should record backup creation, deletion attempts, key usage, retention changes and restores.

Cost advantages over traditional storage:

Cloud backup avoids purchasing large storage arrays upfront and reduces the need to maintain physical capacity for unpredictable growth. Consumption-based pricing allows backup data to move through hot, cool and archival tiers as recovery frequency decreases. Incremental backups, compression and deduplication can reduce transferred and retained bytes where supported.

Lifecycle policies automatically archive or delete data after approved retention periods. However, cloud storage is not automatically cheaper: retrieval fees, minimum storage durations, API requests, network egress, cross-region replication and restore testing must be included in total-cost comparisons. Frequently restored data may belong in a warmer tier despite its higher monthly storage price.

Disaster recovery example and restoration procedure:

Consider a hypothetical college whose administrative database is encrypted by ransomware. Administrators first isolate affected systems, revoke compromised credentials and preserve audit evidence. They identify the latest clean recovery point, provision a separate environment and restore the database plus associated files from protected backups.

Integrity checks compare record counts, key transactions and application behavior before traffic is switched to the restored service. If the approved RPO is 30 minutes and the RTO is four hours, backup frequency and restore procedures must be designed and tested to meet those targets; the

stated numbers are illustrative rather than verified outcomes. After recovery, the team updates access controls, patching and the recovery runbook.

Table 2. Backup threats, protection controls and verification

Risk	Protection	Verification
Disk or zone failure	Durable replicated object storage	Provider configuration and retrieval test
Accidental deletion	Versioning and point-in-time recovery	Recover previous object/database state
Ransomware or admin compromise	Immutable, separate backup vault	Denied deletion test and isolated restore
Sensitive data exposure	Encryption, IAM and audit logs	Access-denial and key-policy checks
Uncontrolled spending	Tiering and retention lifecycle	Storage, retrieval and restore cost review

Conclusion:

Cloud-native backup is most effective when durability is combined with historical versions, independent protected copies, encryption, restricted access and regularly tested restoration. The correct solution optimizes recoverability and security rather than treating redundant storage alone as a disaster recovery plan.

QUESTION 3 – GLOBAL CDN PERFORMANCE AND LOW-LATENCY DELIVERY

Scenario and global application requirements:

A mobile application serving users across continents experiences variable network latency because distant users may need to fetch content from one centralized origin. Network round-trip time, congestion, packet loss and large asset sizes can delay application start-up and interrupt playback. The company needs a global delivery architecture that reduces distance-dependent delays while maintaining secure, consistent content and reliable application APIs. Content delivery networks are particularly valuable for cacheable media, images, scripts, software updates and video segments.

CDN architecture and edge-server operation:

A CDN maintains geographically distributed points of presence that cache selected origin content. When a mobile user requests a cacheable asset, traffic is directed to an appropriate edge location. A cache hit can be served locally, avoiding a round trip to the distant origin. On a cache miss, the edge fetches the object from an origin or regional shield, stores it according to cache policy and returns it to the user. Cache-control headers, versioned URLs, time-to-live policies and selective invalidation help maintain freshness. Private account data and authorization-sensitive responses require careful cache-key separation or should bypass shared public caches.

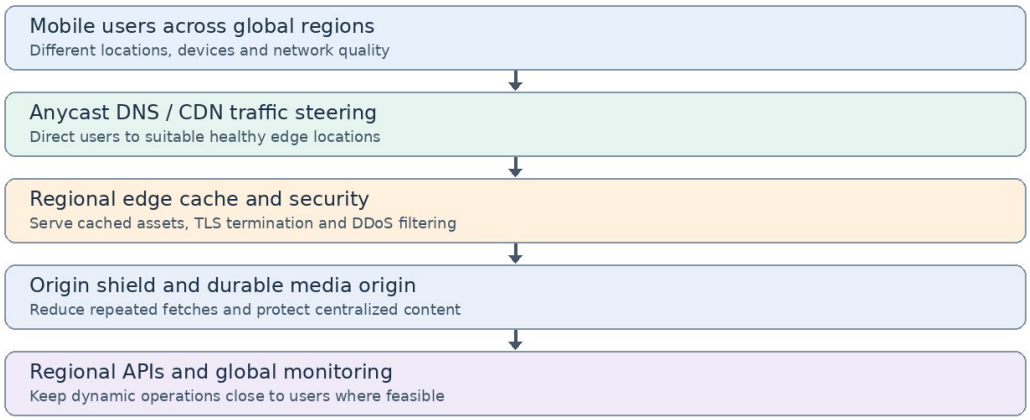


Figure 3. Global CDN design with edge caching, origin protection and regional service placement.

Latency reduction and user experience:

Edge caching reduces repeated transfer of popular objects over long-distance routes. This can improve page rendering, application launch time, video start-up and image loading. Modern protocols such as HTTP/2 or HTTP/3 may further improve connection behavior depending on client and network conditions. Compression and image optimization reduce payload size. For live streaming, adaptive bitrate algorithms select media quality based on observed throughput and buffer conditions, reducing stalls during network variation. A CDN does not eliminate all latency: personalized database queries, game-state updates and cache misses still depend on origin and application design.

Cloud support for global availability:

Cloud providers offer multiple geographic regions, availability zones, managed global load balancing, DNS health checks, CDN services and regional deployment automation. Stateless API services can be replicated across regions, while data stores require an explicit consistency and replication strategy. Health-based traffic routing should redirect users when a region is unavailable, subject to DNS caching and service-specific behavior. Regional data residency, authentication, encryption, observability and capacity must be evaluated before deploying sensitive workloads globally. High availability depends on testing end-to-end application dependencies, not merely enabling a CDN.

Application example for real-time streaming or gaming:

Consider a hypothetical live sports application serving viewers in India, Europe and North America. The encoder generates multiple video renditions and places segments in origin storage. CDN edges distribute those segments near viewers, while regional API endpoints handle login, entitlements and playback authorization. If demand rises during a final match, the CDN absorbs repeated segment requests and reduces origin bandwidth. Player telemetry measures startup delay, rebuffering and playback failures. For a multiplayer game, CDN edges are effective for downloadable game assets, but real-time game state normally needs low-latency regional authoritative servers or specialized edge compute; caching cannot replace interactive synchronization.

Table 3. CDN techniques and global performance measures

Technique	Expected benefit	Evaluation metric
Edge caching	Reduce distance to static assets	Cache hit ratio and download latency
Origin shielding	Reduce origin fetch amplification	Origin request rate and bandwidth
Adaptive bitrate streaming	Reduce video stalls	Rebuffer ratio and startup time
Regional API deployments	Lower dynamic request delay	p95 API latency by region
Health-aware routing	Limit region outage impact	Failover time and request success rate

Conclusion:

A CDN improves global mobile performance by placing cacheable content closer to users and reducing unnecessary origin traffic. Combined with regional APIs, resilient routing and appropriate streaming protocols, edge delivery improves perceived responsiveness and availability while respecting the limits of caching for dynamic or real-time state.

QUESTION 4 – FINANCIAL CLOUD COMPLIANCE, AUDITING AND DATA SOVEREIGNTY

Scenario and regulatory risk interpretation:

A financial institution processes sensitive customer identities, account balances and transaction histories. Cloud adoption must preserve confidentiality, transaction integrity, auditability and service availability while complying with the laws and supervisory requirements applicable to the institution. The assessment does not specify a country or regulator, so no single law can be assumed. The institution must map its actual obligations for data residency, retention, outsourcing, incident reporting, third-party oversight and encryption before choosing a cloud region or provider.

Cloud compliance architecture and shared responsibility:

Cloud providers supply security controls and independent assurance artifacts, such as service certifications and audit reports, but these do not automatically make a customer application compliant. Under the shared-responsibility model, the provider protects the underlying service infrastructure according to the service type, while the financial institution remains responsible for its data classification, user permissions, application configuration and regulatory obligations. A controlled cloud landing zone should enforce approved regions, account separation, private network access, policy-as-code checks, centralized audit collection and continuous configuration monitoring.

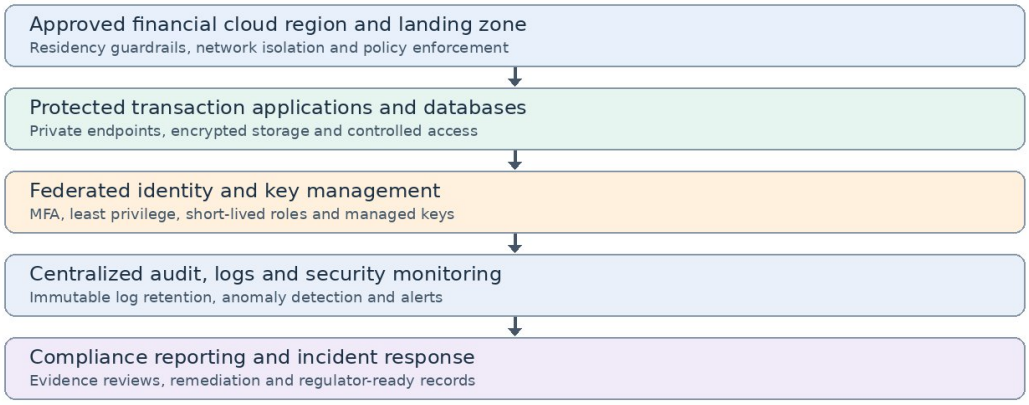


Figure 4. Financial cloud security architecture emphasizing residency, encryption and auditable controls.

Logging, monitoring and encryption controls:

Security logging should record identity authentication, privileged access, API changes, network security events, key operations, database access where supported and financial transaction events. Logs must be timestamped, protected against unauthorized alteration, retained according to policy and accessible only to authorized reviewers. Security monitoring correlates these events to identify unusual logins, unexpected data exports, disabled controls or anomalous payment activity. Encryption in transit protects service communication through modern TLS, while encryption at rest protects databases, object storage and backups. Customer-managed keys or hardware security modules may be appropriate for high-sensitivity operations; key access, rotation, separation of duties and recovery must be designed explicitly.

Data sovereignty and residency enforcement:

The institution should classify datasets and determine which data must remain in a specified jurisdiction. Workload placement policies can restrict resource creation to approved regions. Data flow maps must include primary databases, replicas, backups, monitoring records, support access and disaster recovery copies. A regional cloud endpoint does not alone prove that every copy or support workflow meets sovereignty requirements. The organization should verify provider contractual commitments, data-processing terms and service-specific replication behavior. Periodic audits should compare actual resource inventories and access logs with approved policies.

Example involving sensitive financial transactions:

Consider a hypothetical bank that accepts digital payments through a mobile application. A customer authenticates using strong identity controls, the payment API validates authorization and transaction limits, and the ledger records a durable transaction with idempotency protection against duplicate processing. Sensitive data is encrypted and access is restricted to service identities. Audit records connect the customer request, service action and administrative changes using traceable identifiers without exposing secrets in logs. If monitoring detects an unusual privileged export of account data, the security team investigates, suspends risky credentials and preserves evidence. Any notification or reporting obligation is determined by the bank’s applicable jurisdiction and policies.

Practical compliance verification:

The institution should run regular access reviews, vulnerability assessments, encryption-policy checks, backup restoration drills, log-integrity checks and simulated incident response exercises. A compliance dashboard can report resources outside approved regions, public storage exposure, unencrypted datasets, inactive audit collection and overdue remediation. Controls must be verified continuously because secure configurations can drift after deployment.

Table 4. Financial cloud compliance controls and audit evidence

Requirement	Control	Evidence
Data sovereignty	Approved regions and data-flow inventory	Regional resource and backup-location review
Confidentiality	TLS, at-rest encryption and key governance	Encryption configuration and key audit logs
Access security	Federated IAM, MFA and least privilege	Role review and denied-access tests
Auditability	Centralized tamper-resistant logs	Retention and event-correlation review
Incident readiness	Detection alerts and response runbooks	Incident simulation and corrective-action records

Conclusion:

Secure financial cloud adoption requires coordinated technical controls and institution-specific compliance governance. Regional placement, strong identity, encryption, reliable logging and continuous auditing provide defensible security evidence, while the institution retains responsibility for meeting its actual regulatory obligations.

QUESTION 5 – CONTAINERS, VIRTUAL MACHINES AND KUBERNETES MICROSERVICES

Scenario and deployment objectives:

A development team wants to deploy applications more consistently and scale individual components without provisioning a separate full operating system for every service. Containers package an application with its user-space dependencies and run as isolated processes sharing the host kernel. Virtual machines instead present virtual hardware and typically run independent guest operating systems. Both are useful, but they have different isolation, resource and operational characteristics. The correct choice depends on workload trust, compatibility, portability, security and management requirements.

Container and virtual machine architecture comparison:

A container image includes the application, libraries and runtime components needed in user space. A container runtime launches these images using operating-system mechanisms such as namespaces and control groups on Linux. Because containers share a kernel, they typically start faster and have lower per-instance memory and storage overhead than full VMs. A VM contains a guest OS and is managed through a hypervisor, providing a stronger separation boundary for many multi-tenant threat models and supporting different guest operating systems on the same host.

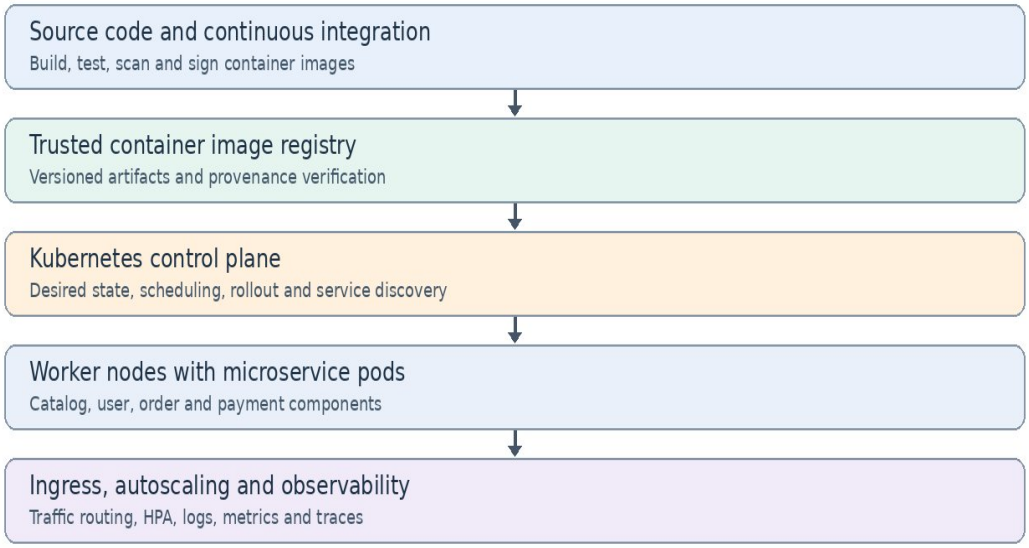


Figure 5. Containerized microservices delivery and Kubernetes orchestration workflow.

How Kubernetes simplifies deployment and scaling:

Kubernetes defines desired application state through declarative resources. Deployments manage replica counts, rolling updates and rollback, while Services provide stable network discovery despite changing pod addresses. Readiness probes prevent traffic reaching unready pods, and liveness probes help restart unhealthy containers. Horizontal Pod Autoscaling adjusts pod replicas using CPU, memory or suitable custom metrics; cluster autoscaling adds nodes when existing capacity cannot schedule more pods. Namespaces, resource quotas, network policies and workload identity support governance. Persistent storage and database consistency still need deliberate architecture, because Kubernetes does not automatically make stateful services highly available.

Portability across cloud environments:

OCI-compatible images and Kubernetes manifests make it easier to deploy similar application workloads on on-premises clusters or managed Kubernetes services across cloud providers. Infrastructure as Code and GitOps can reduce configuration drift and improve reproducibility. Portability is not absolute, however: load balancers, persistent volumes, IAM integration, managed databases and regional networking often differ across providers. A realistic portability strategy separates provider-neutral application configuration from cloud-specific adapters, documents dependencies and tests deployment in each target environment.

Microservices-based application example:

Consider a hypothetical online food delivery service divided into user authentication, restaurant catalog, ordering, payment and notification microservices. Each service is packaged as a container image and deployed independently through Kubernetes. The catalog service may need many replicas during lunch-hour browsing, while the payment service scales according to verified transaction load and maintains stricter security controls.

Kubernetes Services provide internal discovery, an ingress gateway accepts external requests and a message queue decouples notifications from the core order workflow. Database transactions and idempotent payment operations are designed independently of container orchestration. The team uses rolling releases and observability to detect failures before broad deployment.

Resource efficiency and operational risks:

Containers can increase deployment density and reduce startup overhead, but unconstrained resource usage may cause noisy-neighbor effects, out-of-memory terminations and node instability. CPU and memory requests and limits should reflect measured demand. Additional risks include vulnerable base images, excessive privileges, compromised registries, exposed Kubernetes APIs and misconfigured network policies. Secure clusters therefore require admission policies, signed images, secrets management, network segmentation, least privilege, patching and tested recovery procedures.

Table 5. Container, virtual machine and orchestration comparison

Aspect	Containers	Virtual machines / orchestration note
Operating system	Share host kernel	VMs typically have independent guest OS
Startup and overhead	Usually lightweight and fast	VMs usually require more boot time and memory
Isolation boundary	Process/kernel isolation	Hypervisor and guest OS boundary
Scaling	Replicate containers as demand changes	Kubernetes coordinates pods; VM autoscaling is separate
Portability	OCI images ease application packaging	Cloud-specific storage, IAM and networking remain
Primary risk	Kernel and cluster misconfiguration	Hypervisor/guest patching and VM overhead

Conclusion:

Containerization is a strong choice for independently deployable microservices because it supports consistent packaging, rapid startup and granular scaling. Kubernetes adds scheduling, discovery, rollout and health management, while virtual machines remain valuable for stronger workload separation and guest operating-system flexibility. A secure deployment combines both

technologies where appropriate and validates portability rather than assuming complete cloud independence.